Chapter 1 - AWT and Swing

Introduction

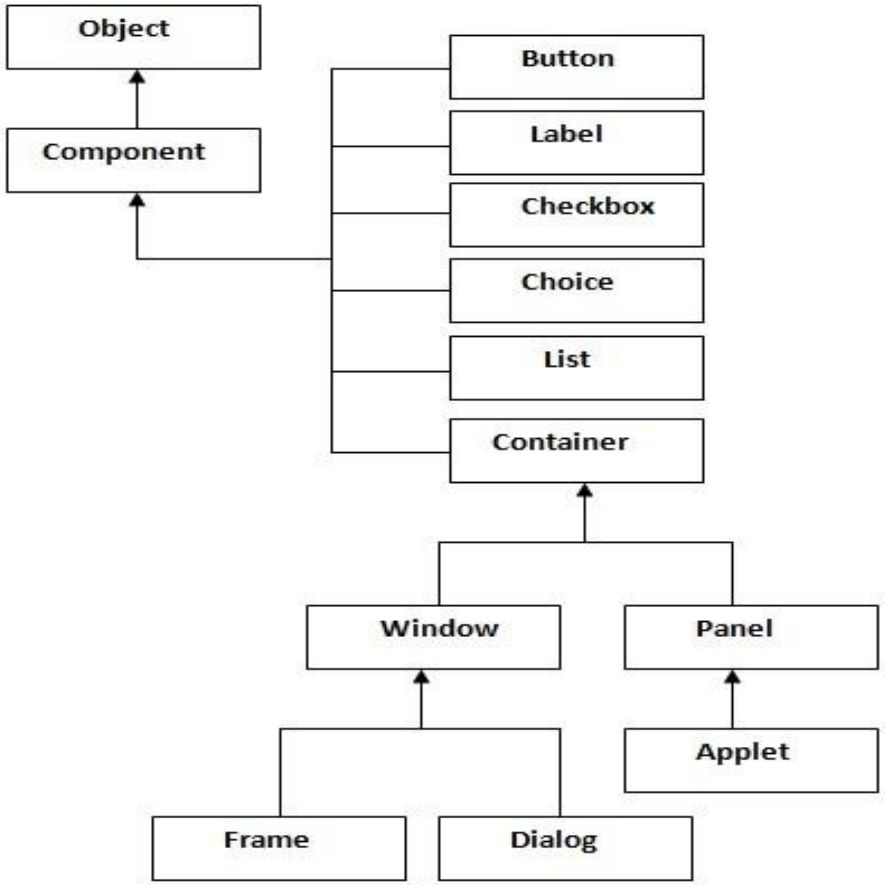
- A graphical user interface (GUI) presents a user-friendly mechanism for interacting with an application. It gives an application a distinctive "look" and "feel."
- A GUI component is an object with which the user interacts via the mouse, the keyboard or another form of input, such as voice recognition.
- **Java GUI APIs:** Java has two GUI packages, the original Abstract Windows Toolkit (AWT) and the newer Swing.

Abstract Windows Toolkit (AWT)

- Is an API to develop GUI or window-based applications in java
- It is Java's original set of classes for building GUIs
- Java AWT components are platform-dependent i.e. components are displayed according to the view of operating system
- AWT is heavyweight i.e. its components are using the resources of OS.
- The java.awt package provides classes for AWT api such as TextField, Label, TextArea, RadioButton, CheckBox, Choice, List etc.

Java AWT Hierarchy

The hierarchy of Java AWT classes are given below



Java Swing

- Is designed to solve AWT's problems (since Java 2)
- It is built on the top of AWT API and entirely written in java
- Unlike AWT, Java Swing provides platform-independent and lightweight components.
- The javax.swing package provides classes for java swing API such as JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser etc.

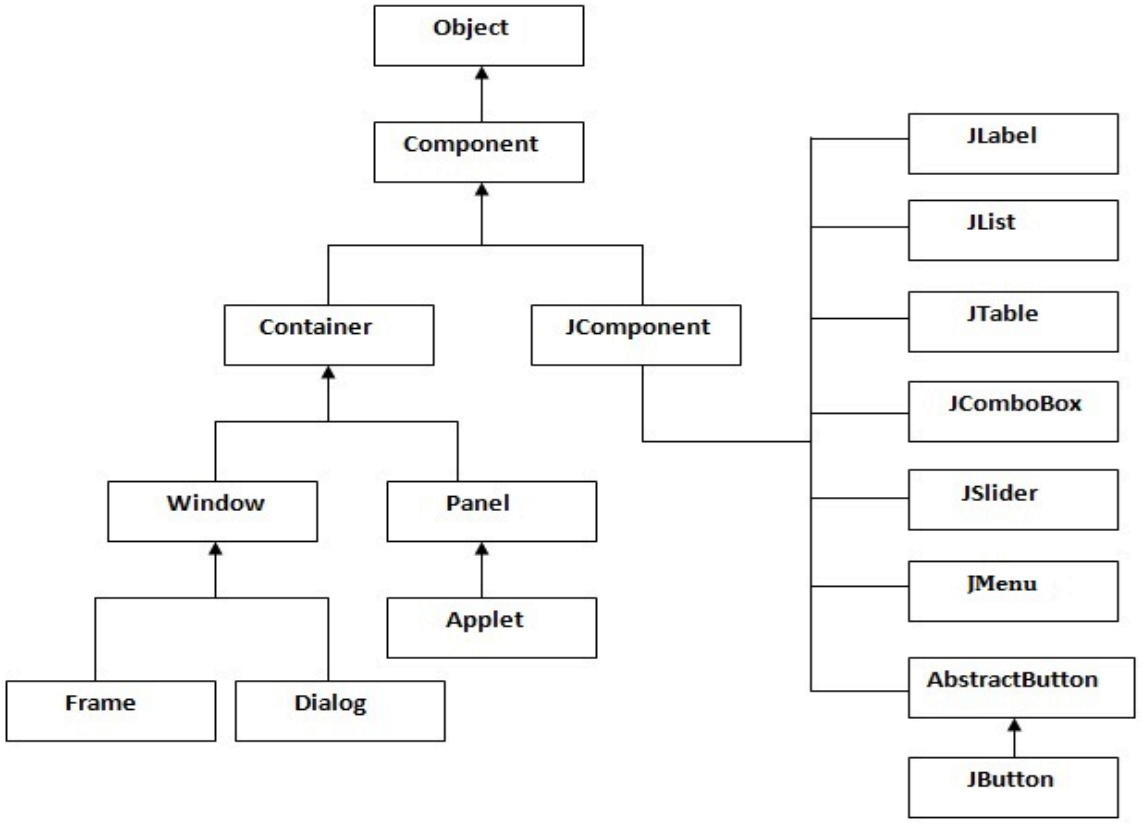
Difference between AWT and Swing

AWT	Swing
Components are platform-dependent	components are platform-independent
Components are heavyweight	components are lightweight
Doesn't support pluggable look and feel	supports pluggable look and feel
provides less components than Swing	provides more powerful components such as tables, lists, scrollpanes, colorchooser, tabbedpane etc
Doesn't follows MVC(Model View Controller)	Swing follows MVC
execution of AWT is slower	Swing executes faster

- ✓ AWT: `Button b=new Button("Click Here");`
- ✓ Swing: `JButton b=new JButton("Click Here");`

Hierarchy of Java Swing

- The hierarchy of java swing API is given below



- Generally, the Java GUI APIs are categorized into three groups. These are:

Container class

Component class

Helper class

1. GUI Container classes

- A GUI is built by putting components/controls into containers.
- Thus, container is used to group components.
- Frames, Panels and applets are examples of containers.
- Important Container classes are JFrame, JApplet, JDialog and JPanel.
- Frame: A resizable, movable window with title bar and close button. Usually it contains JPanels. It is a container that holds other Swing user-interface components in Java GUI application.

Cont….

- JPanel: is a region internal to a JFrame or another JPanel. It is used for grouping components together.
- JPanel is optionally bounded by a visible border . It lives inside some enclosing Container. Panels can be nested, that is, you can place panels inside a container that includes a panel. In fact, every frame has at least one pane, the default "**Content Pane**".

2. GUI Component classes

- GUI Components or Controls (also known as "widgets") are the basic user interface elements the user interacts with.
- These include labels, buttons and text fields. The visual arrangement of the components depends on the container's **layout** (which are part of helper classes).
- Here is a list of controls in the javax.swing package.

  - Input Components
    - Text (JTextField, JTextArea)
    - Menus (JMenuBar, JMenu, JMenuItem)
    - Buttons (JButton, JRadioButtons, JCheckBox)
    - Sliders (JSlider)
    - JComboBox (uneditable) (JComboBox)
    - List (Jlist)

Cont.

- Information Display Components
  - JLabel
  - Tool tips (using JComponent's setToolTipText(s) method)
  - Progress bars (JProgressBar)
- Choosers
  - File chooser (JFileChooser)
  - Color chooser (JColorChooser)
- More complex display
  - Tables (JTable)
  - Trees (JTree)

3. GUI helper classes

- Helper classes are used to describe the properties of GUI components. It encompasses the classes for graphics context, colors, fonts and dimension
- **Graphics**: It is an abstract class that provides a graphical context for drawings strings, lines, and simple shapes.
- **Color**: It deals with the colors of GUI components.
- **Font**: It specifies fonts for the text and drawings on GUI components. Font is available on the **java.awt** package.
- **Layout Manager**: how components are arranged in a container. There are different types of layouts. **FlowLayout**, **GridLayout** and BorderLayout are three examples of the layouts. These layouts are found in the **java.awt** package of JDK API.

Building GUI Application

- In order to build GUI application the component class are commonly used the following methods in java

Method	Description
public void add(Component c)	add a component on another component
public void setSize(int width,int height)	sets size of the component
public void setLayout(LayoutManager m)	sets the layout manager for the component
public void setVisible(boolean b)	sets the visibility of the component. It is by default false

Steps in building GUI Application
1. Creating GUI Containers

- Create a container that could contain components or other containers.
- **Frame**: It is an independent window that has decorations such as a border, a title and buttons for closing, minimizing and maximizing the window.
- Frame is a window that is not contained inside another window. A frame can be moved around on the screen independently of any other GUI windows.
- Applications with a GUI typically use at least one frame.
- Frame is the basis to contain other user interface components in Java GUI applications.

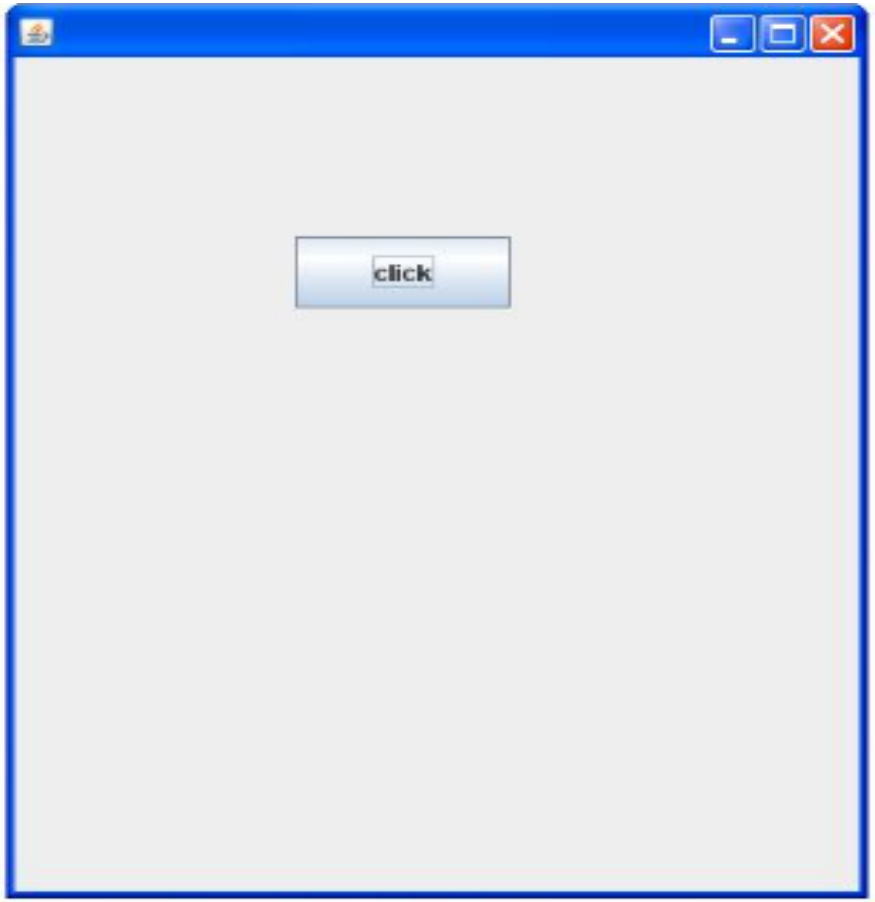
Cont.

- There are two ways to create a frame:
  1. By creating the object of Frame class (association)
  2. By extending Frame class (inheritance)
- We can write the code of swing inside the ***main()***, **constructor** or any **other method**.
- Let's see a simple swing example where we are creating one button and adding it on the JFrame object inside the main() method.

Simple Java Swing Example

File: FirstSwingExample.java

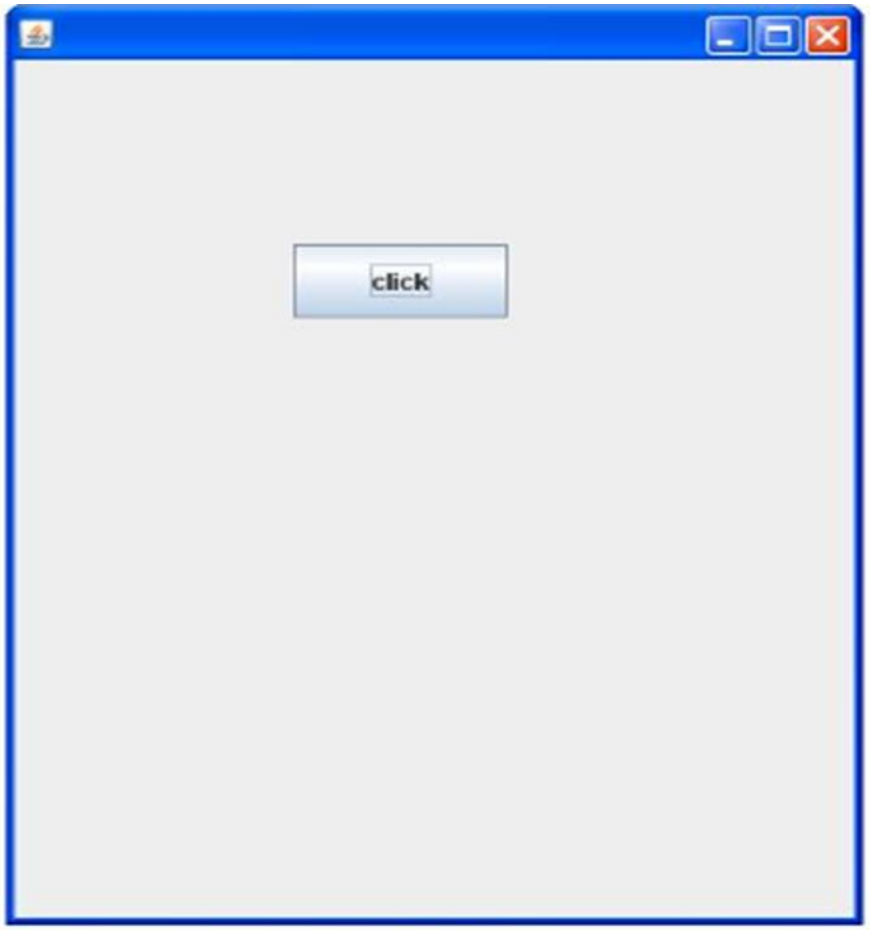
```
import javax.swing.*;
public class FirstSwingExample {
public static void main(String[] args) {
JFrame f=new JFrame();//creating instance of JFrame

JButton b=new JButton("click");//creating instance of JButton
b.setBounds(130,100,100, 40);//x axis, y axis, width, height

f.add(b);//adding button in JFrame

f.setSize(400,500);//400 width and 500 height
f.setLayout(null);//using no layout managers
f.setVisible(true);//making the frame visible
}
}
```



Example of Swing by Association inside constructor

We can also write all the codes of creating JFrame, JButton and method call inside the java constructor

```
import javax.swing.*;
public class Simple {
JFrame f;
Simple(){
f=new JFrame();//creating instance of JFrame

JButton b=new JButton("click");//creating instance of JButton
b.setBounds(130,100,100, 40);

f.add(b);//adding button in JFrame

f.setSize(400,500);//400 width and 500 height
f.setLayout(null);//using no layout managers
f.setVisible(true);//making the frame visible
}

public static void main(String[] args) {
new Simple();
}
}
```



Simple example of Swing by inheritance

We can also inherit the JFrame class, so there is no need to create the instance of JFrame class explicitly

```
import javax.swing.*;
public class Simple2 extends JFrame{//inheriting JFrame
JFrame f;
Simple2(){
JButton b=new JButton("click");//create button
b.setBounds(130,100,100, 40);

add(b);//adding button on frame
setSize(400,500);
setLayout(null);
setVisible(true);
}
public static void main(String[] args) {
new Simple2();
}}
```

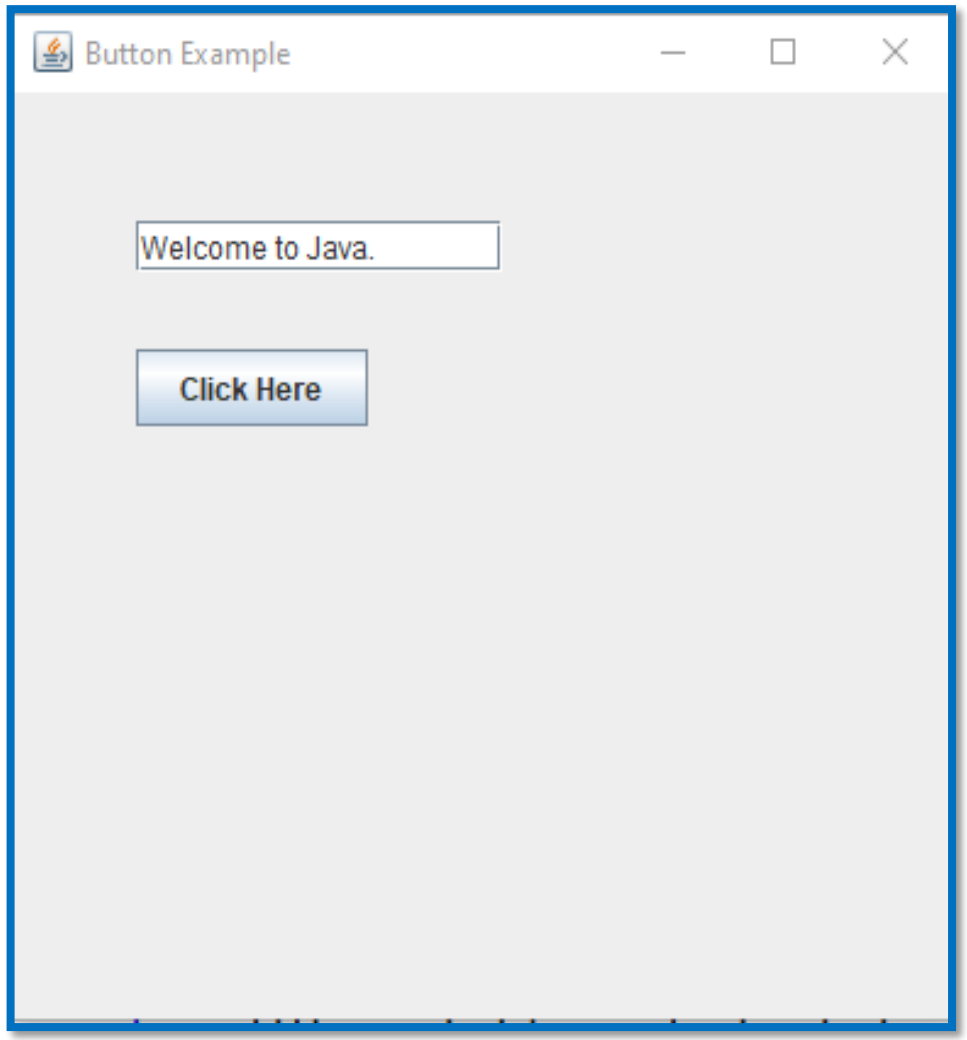
Java JButton

- The **JButton** class is used to create a labeled button that has platform independent implementation. The application result in some action when the button is pushed. It inherits ***AbstractButton*** class.

```
import javax.swing.*;
public class ButtonExample {
public static void main(String[] args) {
    JFrame f=new JFrame("Button Example");
    JButton b=new JButton("Click Here");
    b.setBounds(50,100,95,30);
    f.add(b);
    f.setSize(400,400);
    f.setLayout(null);
    f.setVisible(true);
}
}
```

Java JButton Example with ActionListener

```
import java.awt.event.*;
import javax.swing.*;
public class ButtonExample {
public static void main(String[] args) {
    JFrame f=new JFrame("Button Example");
    final JTextField tf=new JTextField();
    tf.setBounds(50,50, 150,20);
    JButton b=new JButton("Click Here");
    b.setBounds(50,100,95,30);
    b.addActionListener(new ActionListener(){
public void actionPerformed(ActionEvent e){
            tf.setText("Welcome to Javatpoint.");
        }
    });
    f.add(b);f.add(tf);
    f.setSize(400,400);
    f.setLayout(null);
    f.setVisible(true);
}
}
```



Java JLabel

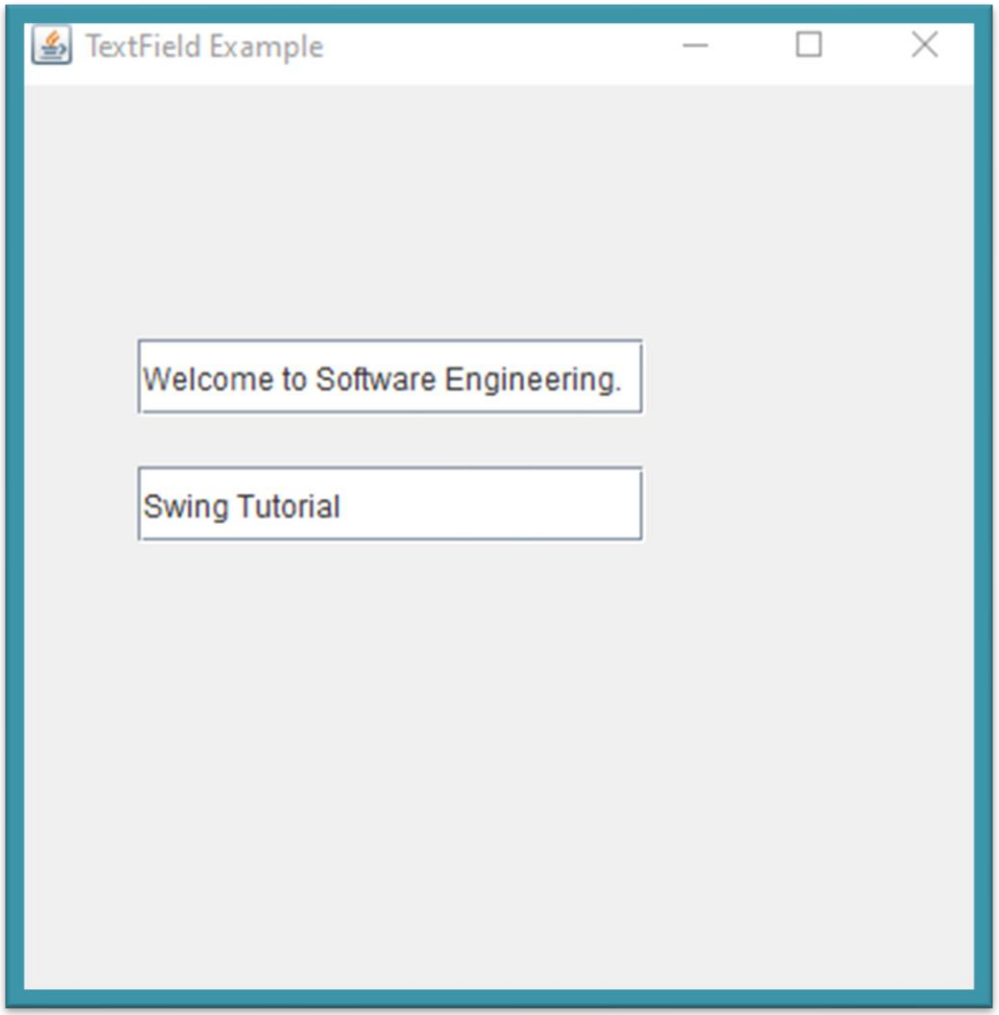
- The object of **JLabel** class is a component for placing text in a container. It is used to display a single line of read only text. The text can be changed by an application but a user cannot edit it directly. It inherits JComponent class.

```
import javax.swing.*;
class LabelExample
{
public static void main(String args[])
    {
    JFrame f= new JFrame("Label Example");
    JLabel l1,l2;
    l1=new JLabel("First Label.");
    l1.setBounds(50,50, 100,30);
    l2=new JLabel("Second Label.");
    l2.setBounds(50,100, 100,30);
    f.add(l1); f.add(l2);
    f.setSize(300,300);
    f.setLayout(null);
    f.setVisible(true);
    }
    }
```

Java JTextField

- The object of a **JTextField** class is a text component that allows the editing of a single line text. It inherits **JTextComponent** class.

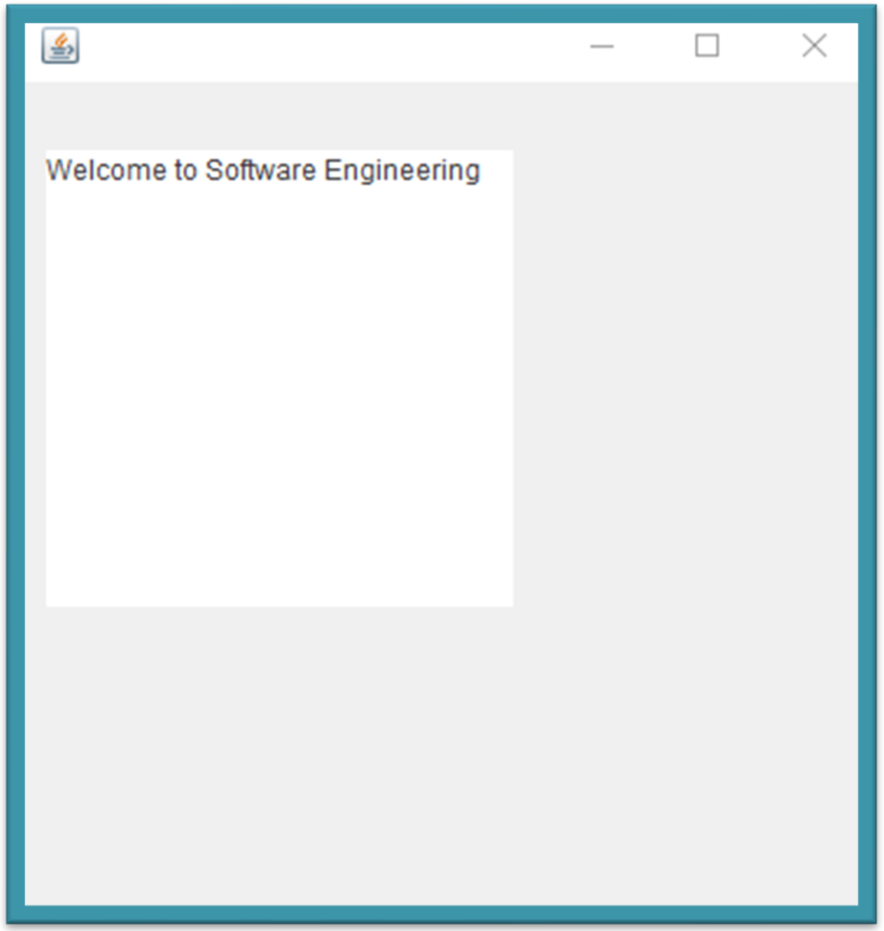
```
import javax.swing.*;
class TextFieldExample
{
public static void main(String args[])
    {
    JFrame f= new JFrame("TextField Example");
    JTextField t1,t2;
    t1=new JTextField("Welcome to Software Engineering.");
    t1.setBounds(50,100, 200,30);
    t2=new JTextField("SwingTutorial");
    t2.setBounds(50,150, 200,30);
    f.add(t1); f.add(t2);
    f.setSize(400,400);
    f.setLayout(null);
    f.setVisible(true);
    }
    }
```



Java JTextArea

- The object of a **JTextArea** class is a multi line region that displays text. It allows the editing of multiple line text. It inherits **JTextComponent** class

```
import javax.swing.*;
public class TextAreaExample
{
    TextAreaExample(){
      JFrame f= new JFrame();
      JTextArea area=new JTextArea("Welcome to Software Engineering");
      area.setBounds(10,30, 200,200);
      f.add(area);
      f.setSize(300,300);
      f.setLayout(null);
      f.setVisible(true);
    }
public static void main(String args[])
   {
  new TextAreaExample();
   }}
```



Java JPasswordField

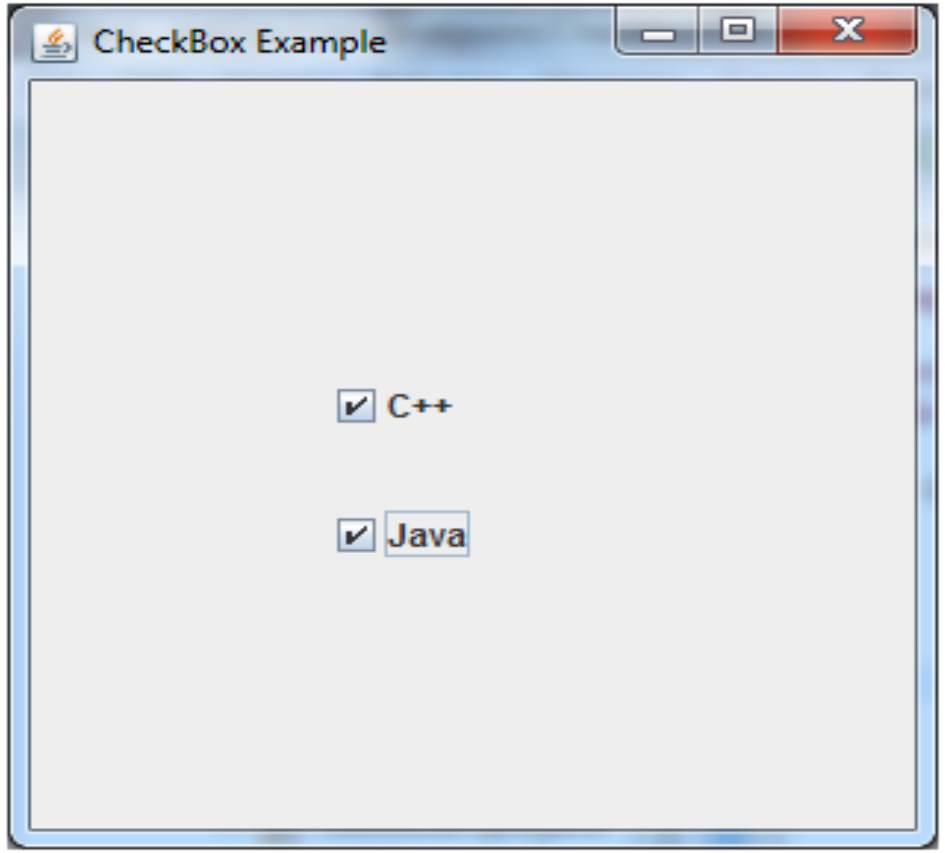
- The object of a **JPasswordField** class is a text component specialized for password entry. It allows the editing of a single line of text. It inherits **JTextField** class.

```
import javax.swing.*;
public class PasswordFieldExample {
    public static void main(String[] args) {
    JFrame f=new JFrame("Password Field Example");
     JPasswordField value = new JPasswordField();
     JLabel l1=new JLabel("Password:");
       l1.setBounds(20,100, 80,30);
        value.setBounds(100,100,100,30);
          f.add(value);  f.add(l1);
          f.setSize(300,300);
          f.setLayout(null);
          f.setVisible(true);
}
}
```

Java JCheckBox

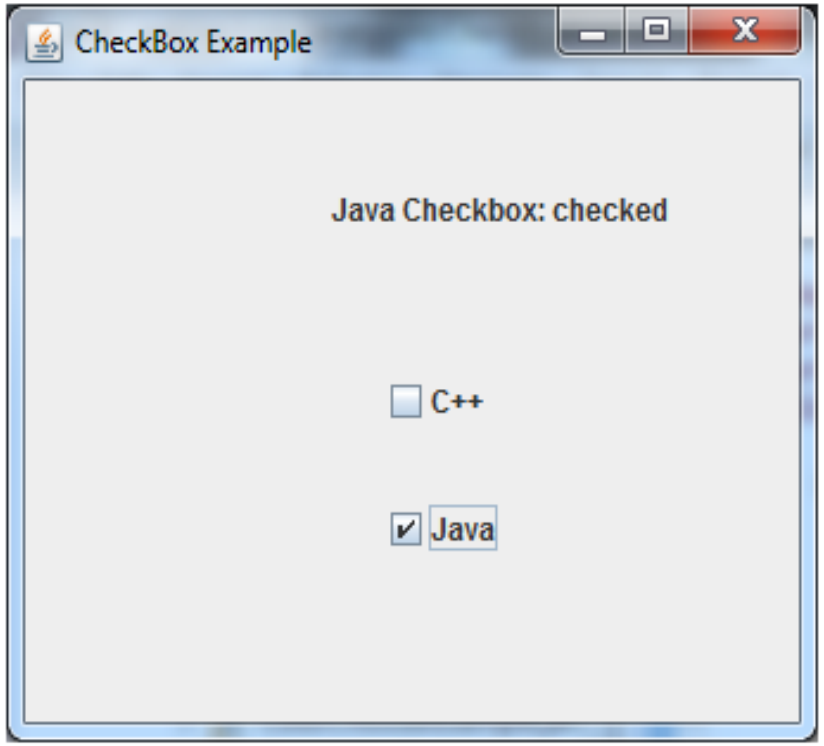
- The **JCheckBox** class is used to create a checkbox. It is used to turn an option on (true) or off (false). Clicking on a **CheckBox** changes its state from "on" to "off" or from "off" to "on ".It inherits JToggleButton class.

```
import javax.swing.*;
public class CheckBoxExample
{
    CheckBoxExample(){
      JFrame f= new JFrame("CheckBox Example");
      JCheckBox checkBox1 = new JCheckBox("C++");
      checkBox1.setBounds(100,100, 50,50);
      JCheckBox checkBox2 = new JCheckBox("Java", true);
      checkBox2.setBounds(100,150, 50,50);
      f.add(checkBox1);
      f.add(checkBox2);
      f.setSize(400,400);
      f.setLayout(null);
      f.setVisible(true);
    }
public static void main(String args[])
   {
   new CheckBoxExample();
   }}
```



Java JCheckBox Example with ItemListener

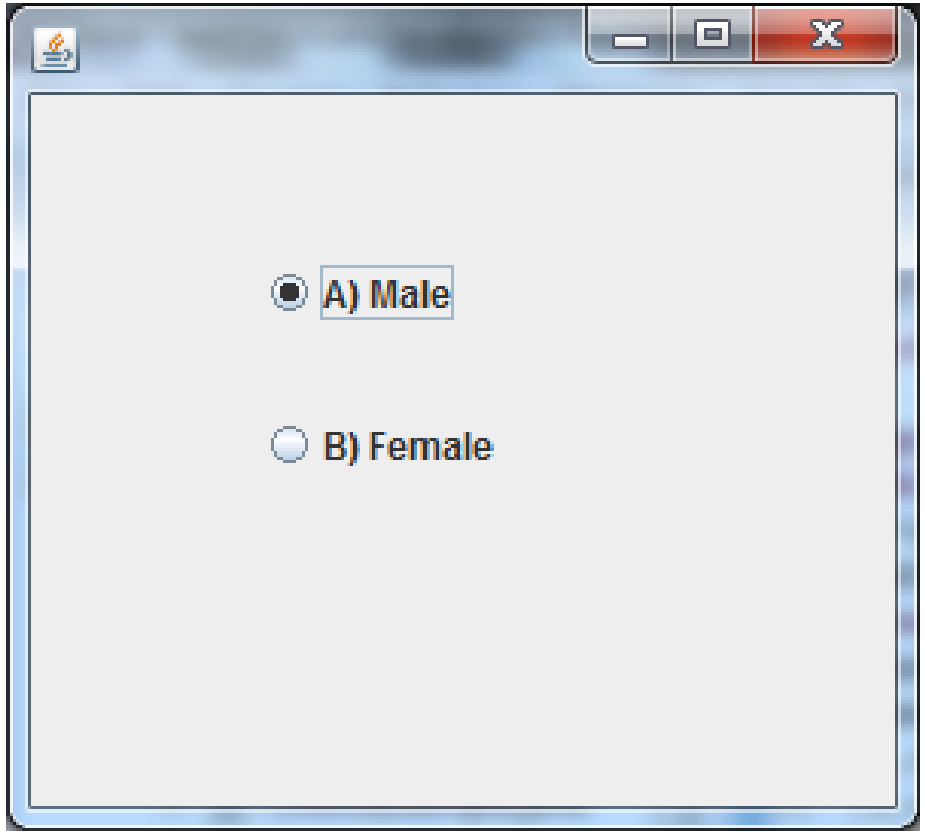
```
import javax.swing.*;
import java.awt.event.*;
public class CheckBoxExample
{
    CheckBoxExample(){
      JFrame f= new JFrame("CheckBox Example");
      final JLabel label = new JLabel();
      label.setHorizontalAlignment(JLabel.CENTER);
      label.setSize(400,100);
      JCheckBox checkbox1 = new JCheckBox("C++");
      checkbox1.setBounds(150,100, 50,50);
      JCheckBox checkbox2 = new JCheckBox("Java");
      checkbox2.setBounds(150,150, 50,50);
      f.add(checkbox1); f.add(checkbox2); f.add(label);
      checkbox1.addItemListener(new ItemListener() {
          public void itemStateChanged(ItemEvent e) {
            label.setText("C++ Checkbox: "
            + (e.getStateChange()==1?"checked":"unchecked"));
          }
       });
       checkbox2.addItemListener(new ItemListener() {
           public void itemStateChanged(ItemEvent e) {
             label.setText("Java Checkbox: "
             + (e.getStateChange()==1?"checked":"unchecked")
           }
         });
       f.setSize(400,400);
       f.setLayout(null);
       f.setVisible(true);
    }
 public static void main(String args[])
 {
    new CheckBoxExample();
 }
 }
```



Java JRadioButton

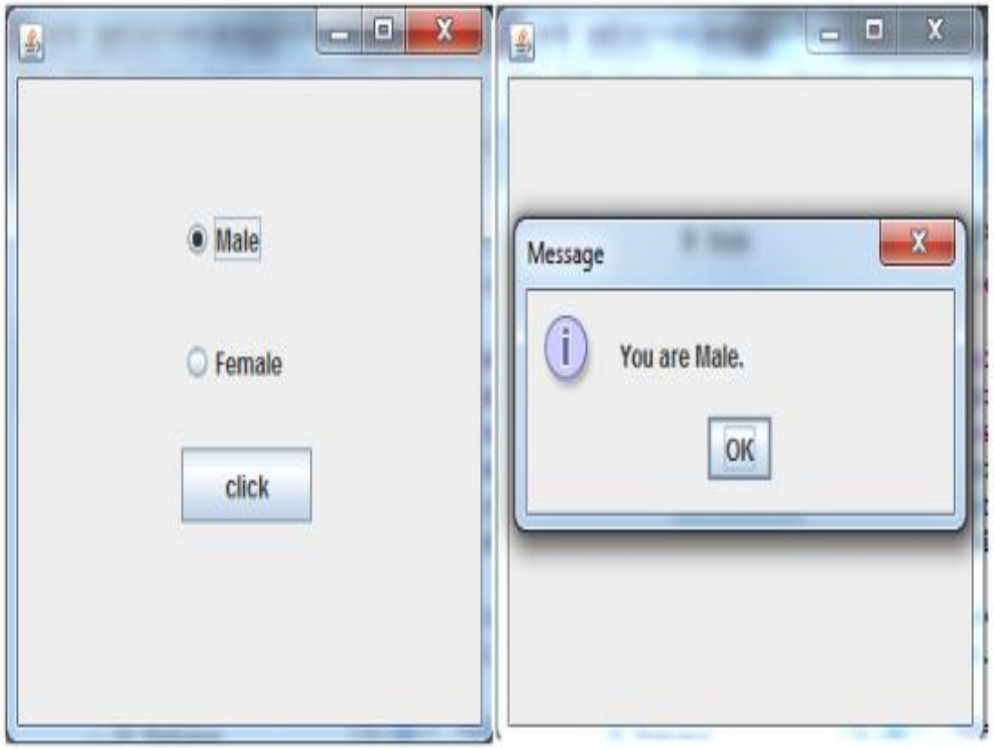
- The JRadioButton class is used to create a radio button. It is used to choose one option from multiple options. It is widely used in exam systems or quiz.
- It should be added in ButtonGroup to select one radio button only.

```
import javax.swing.*;
public class RadioButtonExample {
JFrame f;
RadioButtonExample(){
f=new JFrame();
JRadioButton r1=new JRadioButton("A) Male");
JRadioButton r2=new JRadioButton("B) Female");
r1.setBounds(75,50,100,30);
r2.setBounds(75,100,100,30);
ButtonGroup bg=new ButtonGroup();
bg.add(r1);bg.add(r2);
f.add(r1);f.add(r2);
f.setSize(300,300);
f.setLayout(null);
f.setVisible(true);
}
public static void main(String[] args) {
    new RadioButtonExample();
}
}
```



Java JRadioButton Example with ActionListener

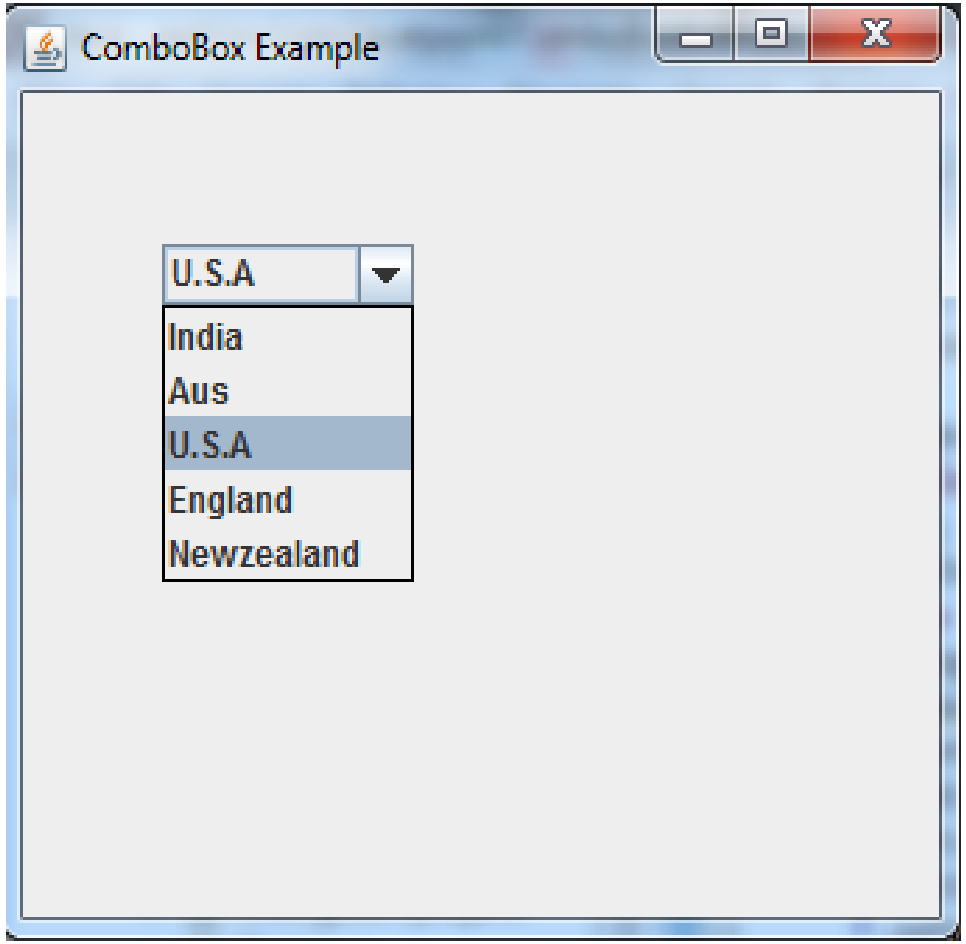
```
import java.awt.event.*;
class RadioButtonExample extends JFrame implements ActionListener{
JRadioButton rb1,rb2;
JButton b;
RadioButtonExample(){
rb1=new JRadioButton("Male");
rb1.setBounds(100,50,100,30);
rb2=new JRadioButton("Female");
rb2.setBounds(100,100,100,30);
ButtonGroup bg=new ButtonGroup();
bg.add(rb1);bg.add(rb2);
b=new JButton("click");
b.setBounds(100,150,80,30);
b.addActionListener(this);
add(rb1);add(rb2);add(b);
setSize(300,300);
setLayout(null);
setVisible(true);
}
public void actionPerformed(ActionEvent e){
if(rb1.isSelected()){
JOptionPane.showMessageDialog(this,"You are Male.");
}
if(rb2.isSelected()){
JOptionPane.showMessageDialog(this,"You are Female.");
}
}
public static void main(String args[]){
new RadioButtonExample();
}}
```



Java JComboBox

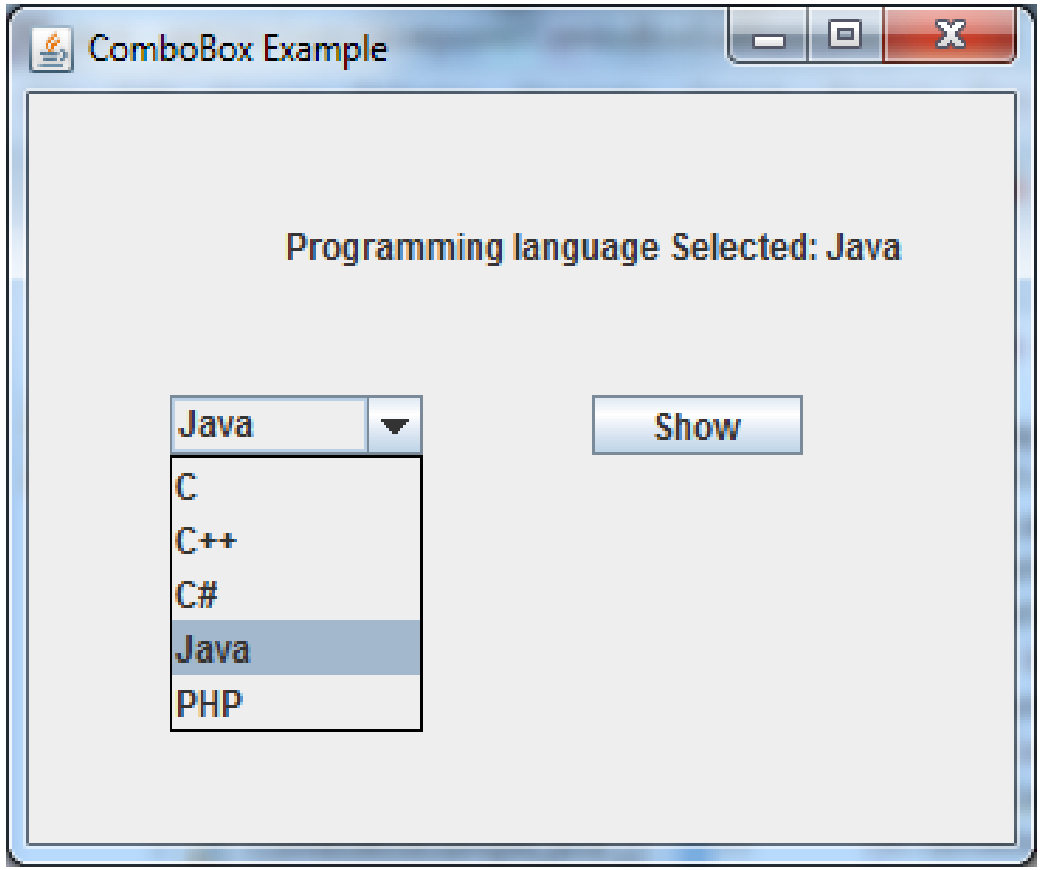
- The object of **Choice** class is used to show popup menu of choices. Choice selected by user is shown on the top of a menu. It inherits JComponent class.

```
import javax.swing.*;
public class ComboBoxExample {
JFrame f;
ComboBoxExample(){
    f=new JFrame("ComboBox Example");
    String country[]={"India","Aus","U.S.A","England","Newzealand"};
    JComboBox cb=new JComboBox(country);
    cb.setBounds(50, 50,90,20);
    f.add(cb);
    f.setLayout(null);
    f.setSize(400,500);
    f.setVisible(true);
}
public static void main(String[] args) {
    new ComboBoxExample();
}
}
```



Java JComboBox Example with ActionListener

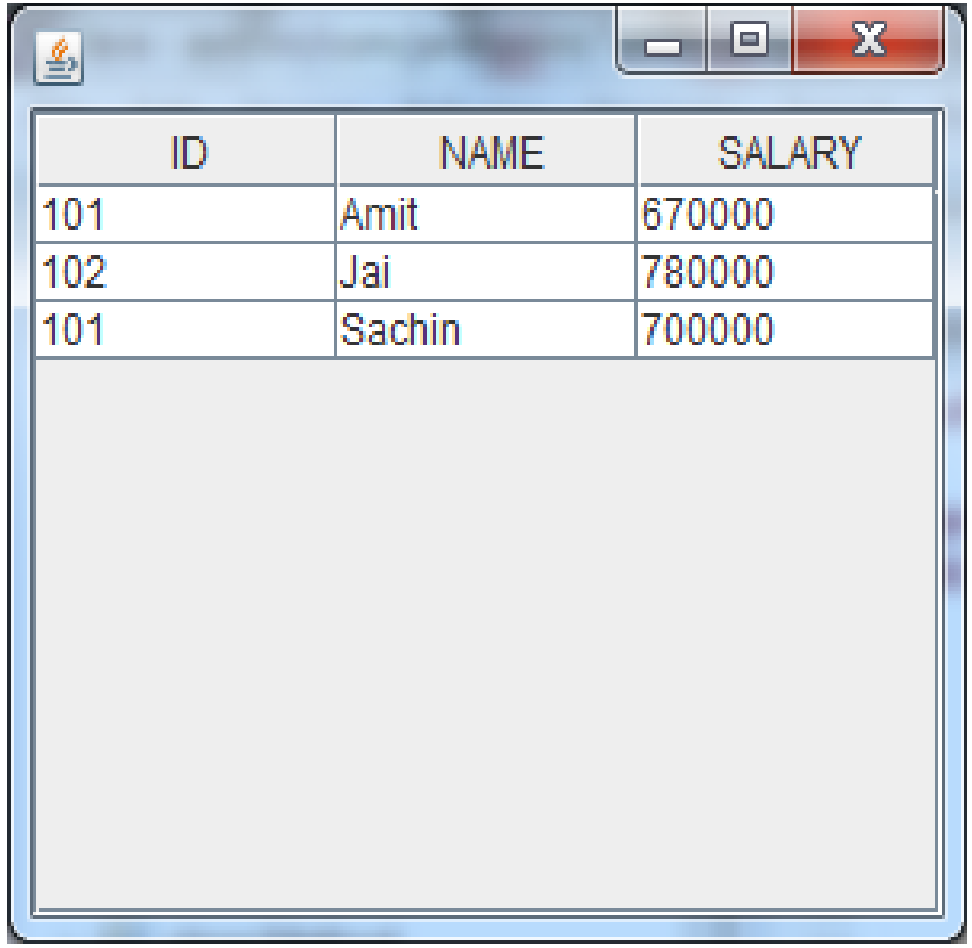
```
import javax.swing.*;
import java.awt.event.*;
public class ComboBoxExample {
JFrame f;
ComboBoxExample(){
   f=new JFrame("ComboBox Example");
   final JLabel label = new JLabel();
   label.setHorizontalAlignment(JLabel.CENTER);
   label.setSize(400,100);
   JButton b=new JButton("Show");
   b.setBounds(200,100,75,20);
   String languages[]={"C","C++","C#","Java","PHP"};
   final JComboBox cb=new JComboBox(languages);
   cb.setBounds(50, 100,90,20);
   f.add(cb); f.add(label); f.add(b);
   f.setLayout(null);
   f.setSize(350,350);
   f.setVisible(true);
   b.addActionListener(new ActionListener() {
      public void actionPerformed(ActionEvent e) {
String data = "Programming language Selected: "

   + cb.getItemAt(cb.getSelectedIndex());
label.setText(data);
}
});
}
public static void main(String[] args) {
   new ComboBoxExample();
}
}
```



Java JTable

- The JTable class is used to display data in tabular form. It is composed of rows and columns.

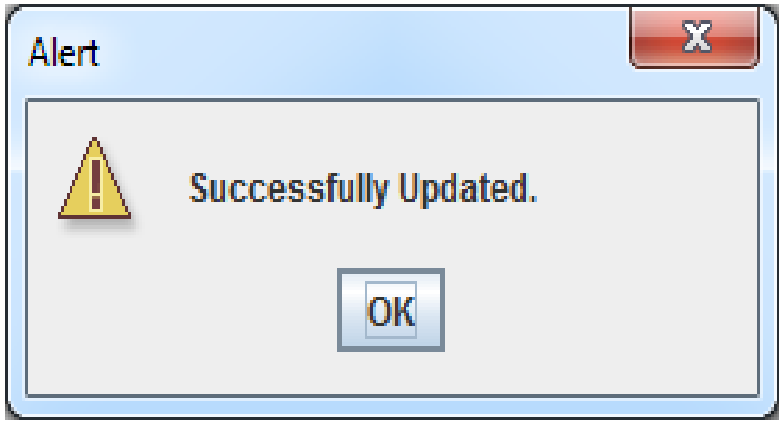
```
import javax.swing.*;
public class TableExample {
    JFrame f;
    TableExample(){
    f=new JFrame();
    String data[][]={ {"101","Amit","670000"},
                      {"102","Jai","780000"},
                      {"101","Sachin","700000"}};
    String column[]={"ID","NAME","SALARY"};
    JTable jt=new JTable(data,column);
    jt.setBounds(30,40,200,300);
    JScrollPane sp=new JScrollPane(jt);
    f.add(sp);
    f.setSize(300,400);
    f.setVisible(true);
}
public static void main(String[] args) {
    new TableExample();
}
}
```



Java JOptionPane

- The **JOptionPane** class is used to provide standard dialog boxes such as message dialog box, confirm dialog box and input dialog box. These dialog boxes are used to display information or get input from the user.

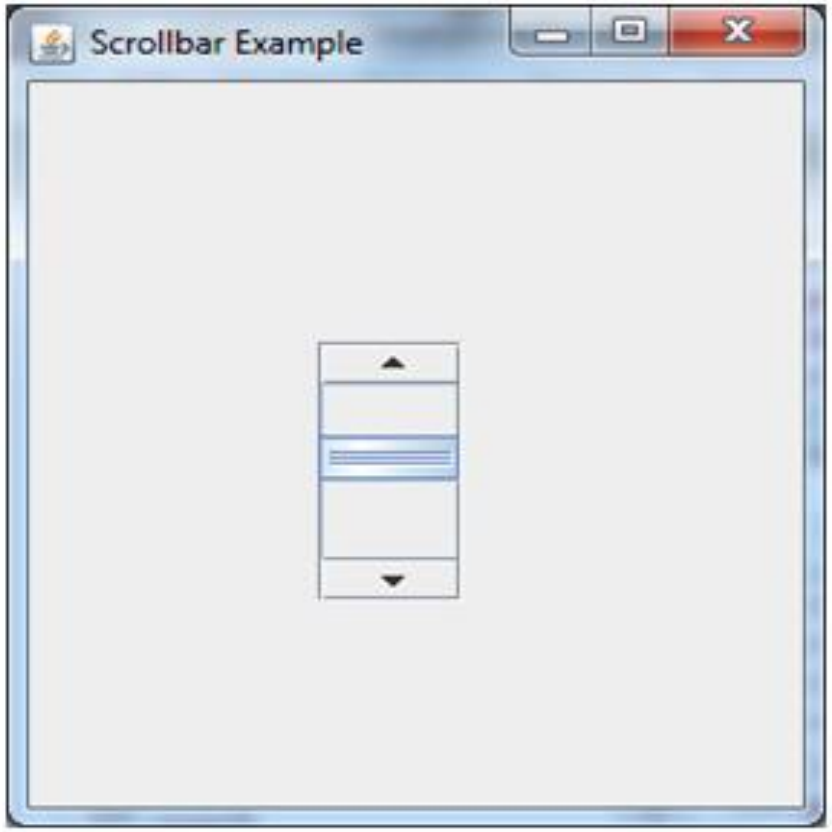
```
import javax.swing.*;
public class OptionPaneExample {
JFrame f;
OptionPaneExample(){
   f=new JFrame();
   JOptionPane.showMessageDialog(f,"Successfully Updated.","Alert",JOptionPane.WARNING_MESSAGE);
}
public static void main(String[] args) {
   new OptionPaneExample();
}
}
```



Java JScrollBar

- The object of **JScrollbar** class is used to add horizontal and vertical scrollbar. It is an implementation of a scrollbar. It inherits **JComponent** class.

```
import javax.swing.*;
class ScrollBarExample
{
ScrollBarExample(){
   JFrame f= new JFrame("Scrollbar Example");
 JScrollBar s=new JScrollBar();
s.setBounds(100,100, 50,100);
f.add(s);
f.setSize(400,400);
f.setLayout(null);
f.setVisible(true);
}
public static void main(String args[])
{
new ScrollBarExample();
}}
```

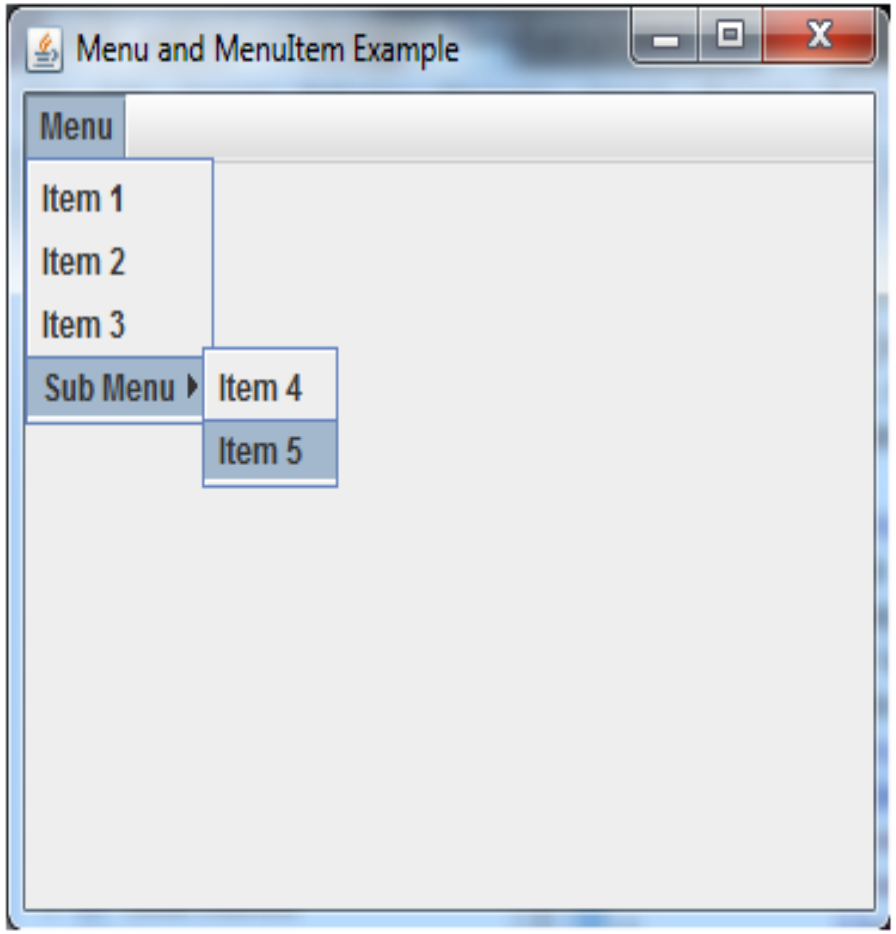


Java JMenuBar, JMenu and JMenuItem

- The **JMenuBar** class is used to display menubar on the window or frame. It may have several menus.
- The object of **JMenu** class is a pull down menu component which is displayed from the menu bar. It inherits the **JMenuItem** class.
- The object of **JMenuItem** class adds a simple labeled menu item. The items used in a menu must belong to the **JMenuItem** or any of its subclass.

Java JMenuItem and JMenu Example

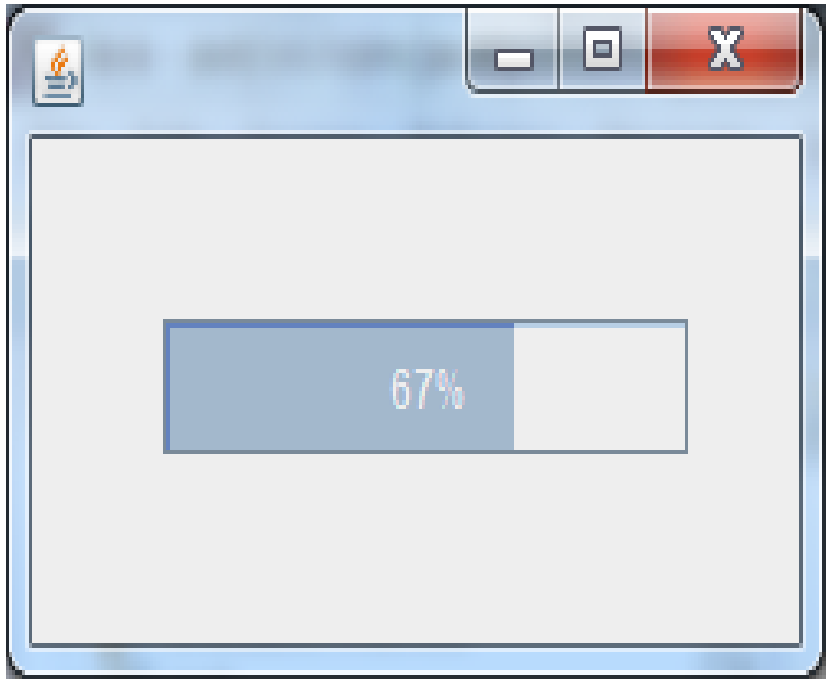
```
class MenuExample
{
        JMenu menu, submenu;
        JMenuItem i1, i2, i3, i4, i5;
        MenuExample(){
        JFrame f= new JFrame("Menu and MenuItem Example");
        JMenuBar mb=new JMenuBar();
        menu=new JMenu("Menu");
        submenu=new JMenu("Sub Menu");
        i1=new JMenuItem("Item 1");
        i2=new JMenuItem("Item 2");
        i3=new JMenuItem("Item 3");
        i4=new JMenuItem("Item 4");
        i5=new JMenuItem("Item 5");
        menu.add(i1); menu.add(i2); menu.add(i3);
        submenu.add(i4); submenu.add(i5);
        menu.add(submenu);
        mb.add(menu);
        f.setJMenuBar(mb);
        f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
}
public static void main(String args[])
{
new MenuExample();
}}
```



Java JProgressBar

- The **JProgressBar** class is used to display the progress of the task. It inherits JComponent class.

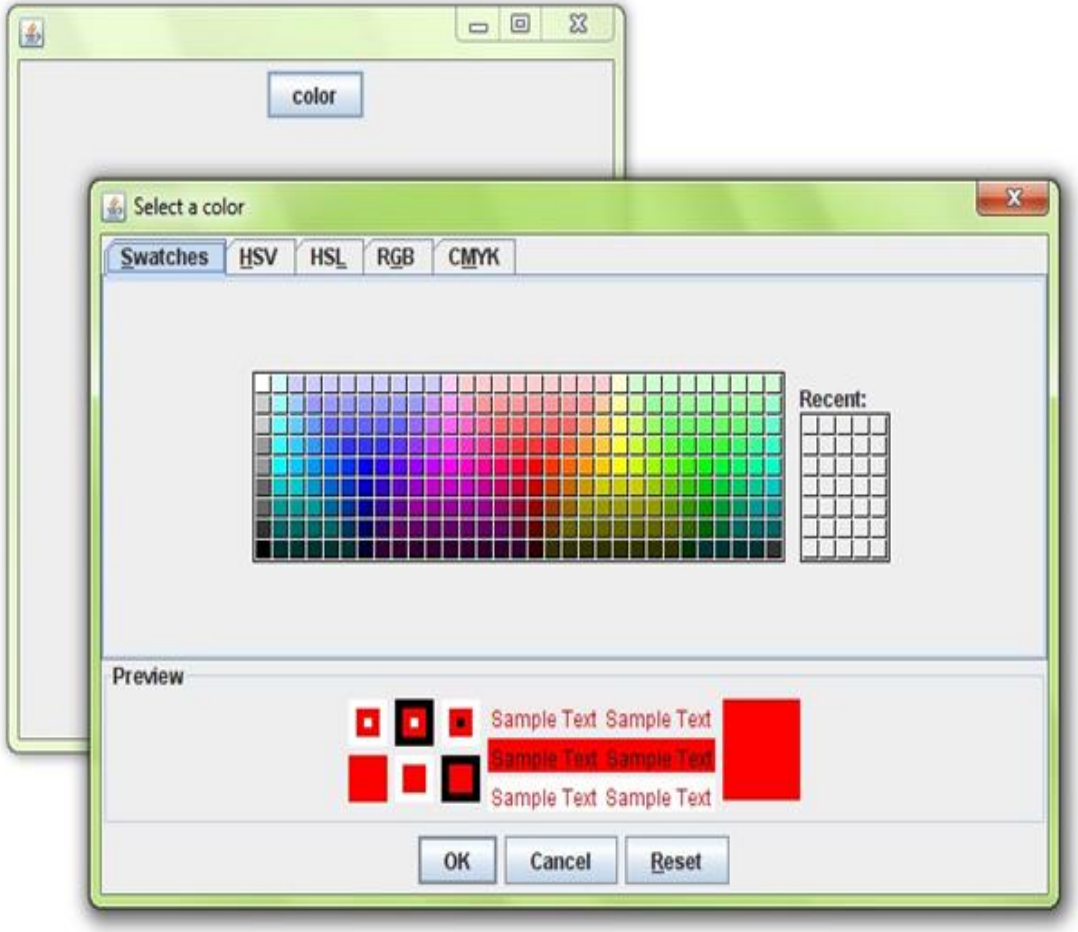
```
import javax.swing.*;
public class ProgressBarExample extends JFrame{
JProgressBar jb;
int i=0,num=0;
ProgressBarExample(){
jb=new JProgressBar(0,2000);
jb.setBounds(40,40,160,30);
jb.setValue(0);
jb.setStringPainted(true);
add(jb);
setSize(250,150);
setLayout(null);
}
public void iterate(){
while(i<=2000){
 jb.setValue(i);
 i=i+20;
 try{Thread.sleep(150);}catch(Exception e){}
}
}
public static void main(String[] args) {
   ProgressBarExample m=new ProgressBarExample();
   m.setVisible(true);
   m.iterate();
}
}
```



Java JColorChooser

- The **JColorChooser** class is used to create a color chooser dialog box so that user can select any color. It inherits JComponent class.

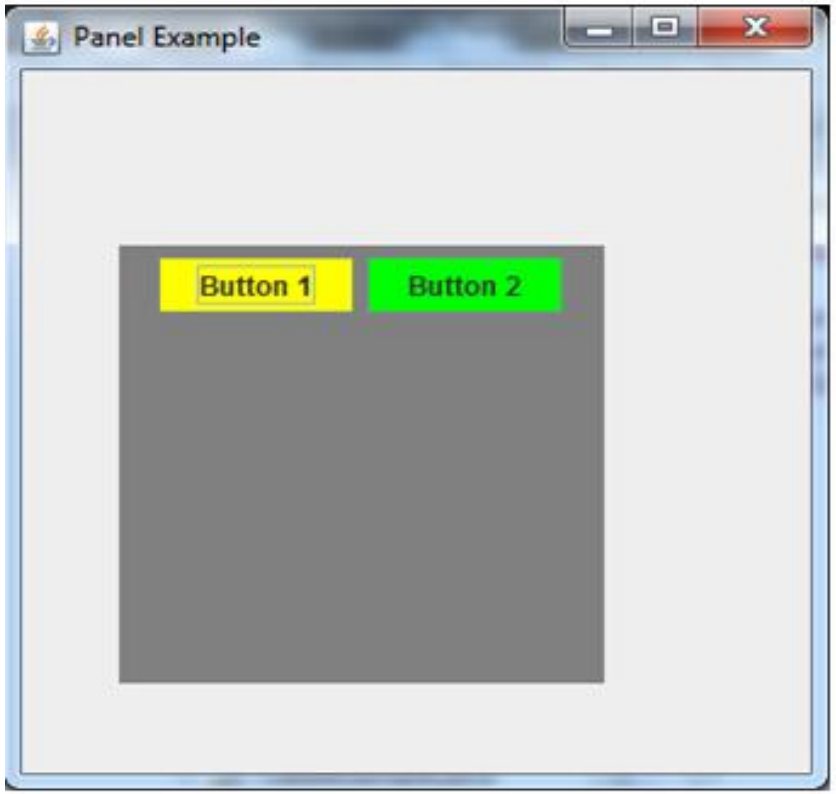
```
import java.awt.event.*;
import java.awt.*;
import javax.swing.*;
public class ColorChooserExample extends JFrame implements ActionListener {
JButton b;
Container c;
ColorChooserExample(){
    c=getContentPane();
    c.setLayout(new FlowLayout());
    b=new JButton("color");
    b.addActionListener(this);
    c.add(b);
}
public void actionPerformed(ActionEvent e) {
Color initialcolor=Color.RED;
Color color=JColorChooser.showDialog(this,"Select a color",initialcolor);
c.setBackground(color);
}

public static void main(String[] args) {
    ColorChooserExample ch=new ColorChooserExample();
    ch.setSize(400,400);
    ch.setVisible(true);
    ch.setDefaultCloseOperation(EXIT_ON_CLOSE);
}
}
```



Java JPanel

- The **JPanel** is a simplest container class. It provides space in which an application can attach any other component. It inherits the JComponents class.It doesn't have title bar.

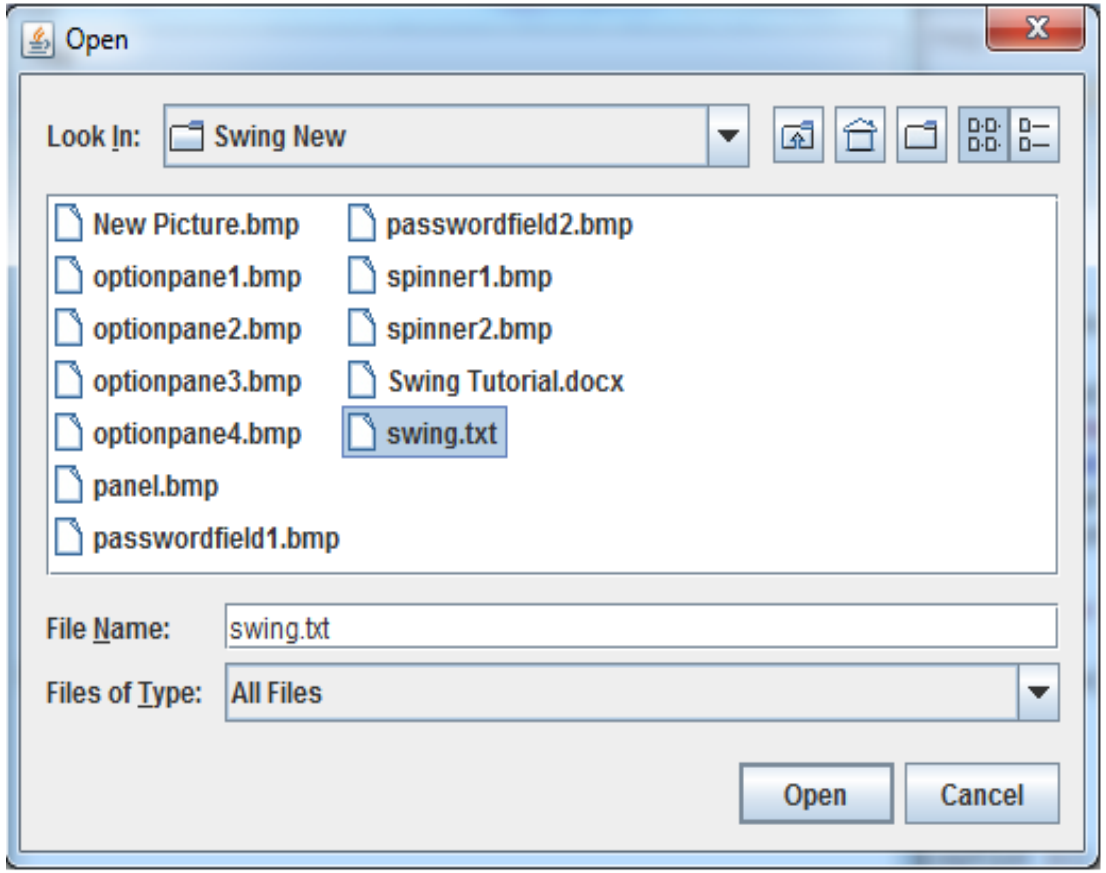
```
import java.awt.*;
import javax.swing.*;
public class PanelExample {
    PanelExample()
      {
      JFrame f= new JFrame("Panel Example");
      JPanel panel=new JPanel();
      panel.setBounds(40,80,200,200);
      panel.setBackground(Color.gray);
      JButton b1=new JButton("Button 1");
      b1.setBounds(50,100,80,30);
      b1.setBackground(Color.yellow);
      JButton b2=new JButton("Button 2");
      b2.setBounds(100,100,80,30);
      b2.setBackground(Color.green);
      panel.add(b1); panel.add(b2);
      f.add(panel);
            f.setSize(400,400);
            f.setLayout(null);
            f.setVisible(true);
      }
      public static void main(String args[])
      {
      new PanelExample();
      }
  }
```



Java JFileChooser

- The object of **JFileChooser** class represents a dialog window from which the user can select file. It inherits JComponent class.

```
public void actionPerformed(ActionEvent e) {
if(e.getSource()==open){
   JFileChooser fc=new JFileChooser();
   int i=fc.showOpenDialog(this);
   if(i==JFileChooser.APPROVE_OPTION){
      File f=fc.getSelectedFile();
      String filepath=f.getPath();
      try{
      BufferedReader br=new BufferedReader(new FileReader(filepath));
      String s1="",s2="";
      while((s1=br.readLine())!=null){
      s2+=s1+"\n";
      }
      ta.setText(s2);
      br.close();
      }catch (Exception ex) {ex.printStackTrace();  }
   }
}
```



Java LayoutManagers

- The **LayoutManagers** are used to arrange components in a particular manner.
- LayoutManager is an interface that is implemented by all the classes of layout managers.
- There are following classes that represents the layout managers:
    1. java.awt.BorderLayout
    2. java.awt.GridLayout
    3. java.awt. CardLayout

Java BorderLayout

- The BorderLayout is used to arrange the components in five regions: **north**, south, east, west and center. Each region (area) may contain one component only. It is the default layout of frame or window.
- The BorderLayout provides five constants for each region:

```
public static final int NORTH
public static final int SOUTH
public static final int EAST
public static final int WEST
public static final int CENTER
```

- **BorderLayout():** creates a border layout but with no gaps between the components.
- **JBorderLayout(int hgap, int vgap):** creates a border layout with the given horizontal and vertical gaps between the components.

Example of BorderLayout class

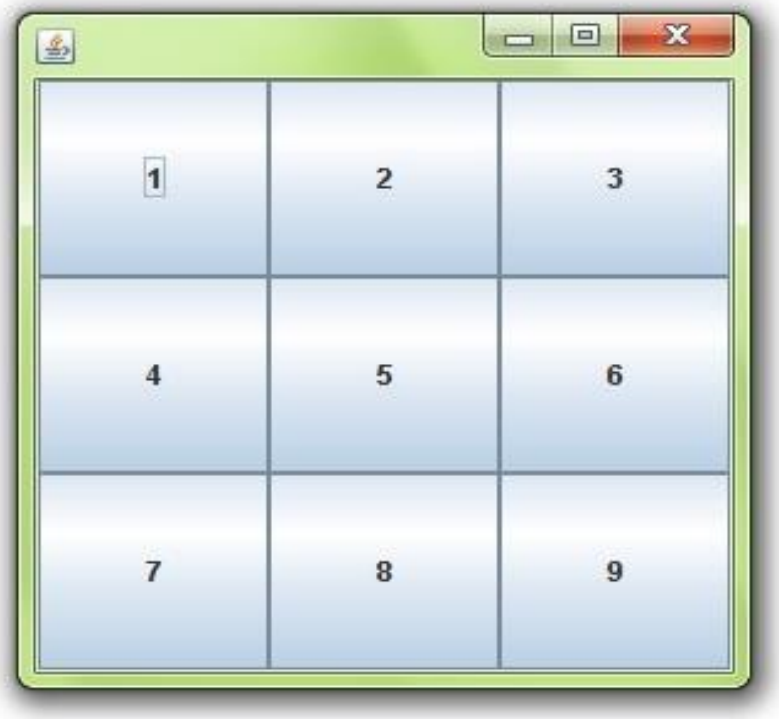
```
import java.awt.*;
import javax.swing.*;

public class Border {
JFrame f;
Border(){
   f=new JFrame();

   JButton b1=new JButton("NORTH");;
   JButton b2=new JButton("SOUTH");;
   JButton b3=new JButton("EAST");;
   JButton b4=new JButton("WEST");;
   JButton b5=new JButton("CENTER");;

   f.add(b1,BorderLayout.NORTH);
   f.add(b2,BorderLayout.SOUTH);
   f.add(b3,BorderLayout.EAST);
   f.add(b4,BorderLayout.WEST);
   f.add(b5,BorderLayout.CENTER);

   f.setSize(300,300);
   f.setVisible(true);
}
public static void main(String[] args) {
   new Border();
}
}
```

Java GridLayout

- The GridLayout is used to arrange the components in rectangular grid. One component is displayed in each rectangle.
- **GridLayout():** creates a grid layout with one column per component in a row.
- **GridLayout(int rows, int columns):** creates a grid layout with the given rows and columns but no gaps between the components.
- **GridLayout(int rows, int columns, int hgap, int vgap):** creates a grid layout with the given rows and columns along with given horizontal and vertical gaps.

Example of GridLayout class

```
public class MyGridLayout{
JFrame f;
MyGridLayout(){
   f=new JFrame();

   JButton b1=new JButton("1");
   JButton b2=new JButton("2");
   JButton b3=new JButton("3");
   JButton b4=new JButton("4");
   JButton b5=new JButton("5");
      JButton b6=new JButton("6");
      JButton b7=new JButton("7");
   JButton b8=new JButton("8");
      JButton b9=new JButton("9");

   f.add(b1);f.add(b2);f.add(b3);f.add(b4);f.add(b5);
   f.add(b6);f.add(b7);f.add(b8);f.add(b9);

   f.setLayout(new GridLayout(3,3));
   //setting grid layout of 3 rows and 3 columns

   f.setSize(300,300);
   f.setVisible(true);
}
public static void main(String[] args) {
   new MyGridLayout();
```

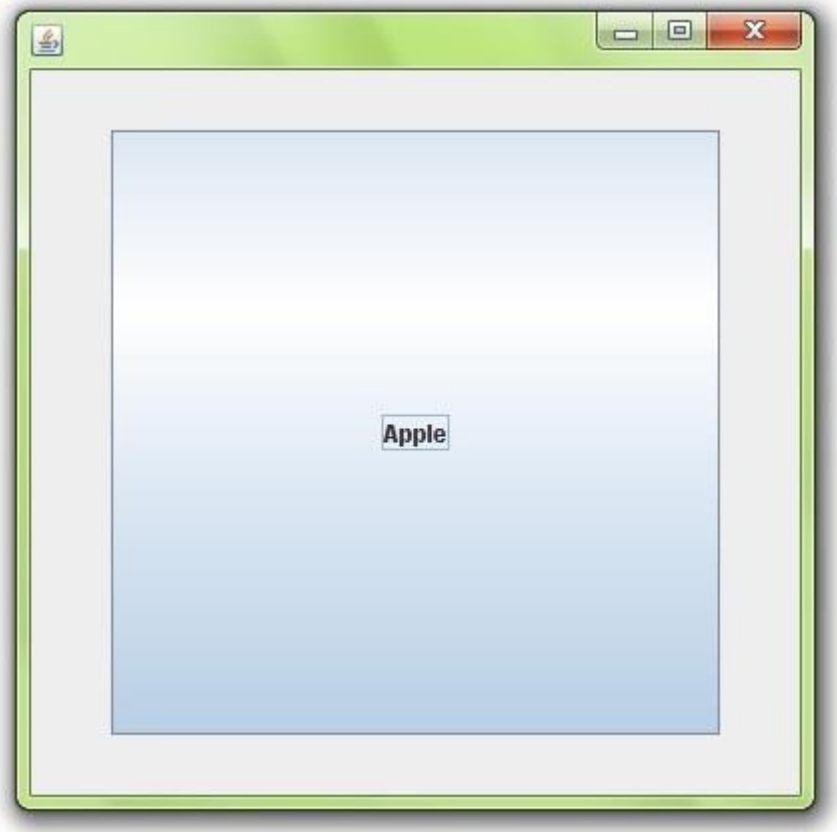


Java CardLayout

- The **CardLayout** class manages the components in such a manner that only one component is visible at a time.
- It treats each component as a card that is why it is known as CardLayout.
- **CardLayout():** creates a card layout with zero horizontal and vertical gap.
- **CardLayout(int hgap, int vgap):** creates a card layout with the given horizontal and vertical gap.

Example of CardLayout class

```
import java.awt.*;
import java.awt.event.*;

import javax.swing.*;

public class CardLayoutExample extends JFrame implements ActionListener{
CardLayout card;
JButton b1,b2,b3;
Container c;
   CardLayoutExample(){

      c=getContentPane();
      card=new CardLayout(40,30);
//create CardLayout object with 40 hor space and 30 ver space
      c.setLayout(card);

      b1=new JButton("Apple");
      b2=new JButton("Boy");
      b3=new JButton("Cat");
      b1.addActionListener(this);
      b2.addActionListener(this);
      b3.addActionListener(this);

      c.add("a",b1);c.add("b",b2);c.add("c",b3);

   }
   public void actionPerformed(ActionEvent e) {
   card.next(c);
   }

   public static void main(String[] args) {
      CardLayoutExample cl=new CardLayoutExample();
      cl.setSize(400,400);
      cl.setVisible(true);
      cl.setDefaultCloseOperation(EXIT_ON_CLOSE);
   }
}
```



Thank you!!

Feel free to ask!!